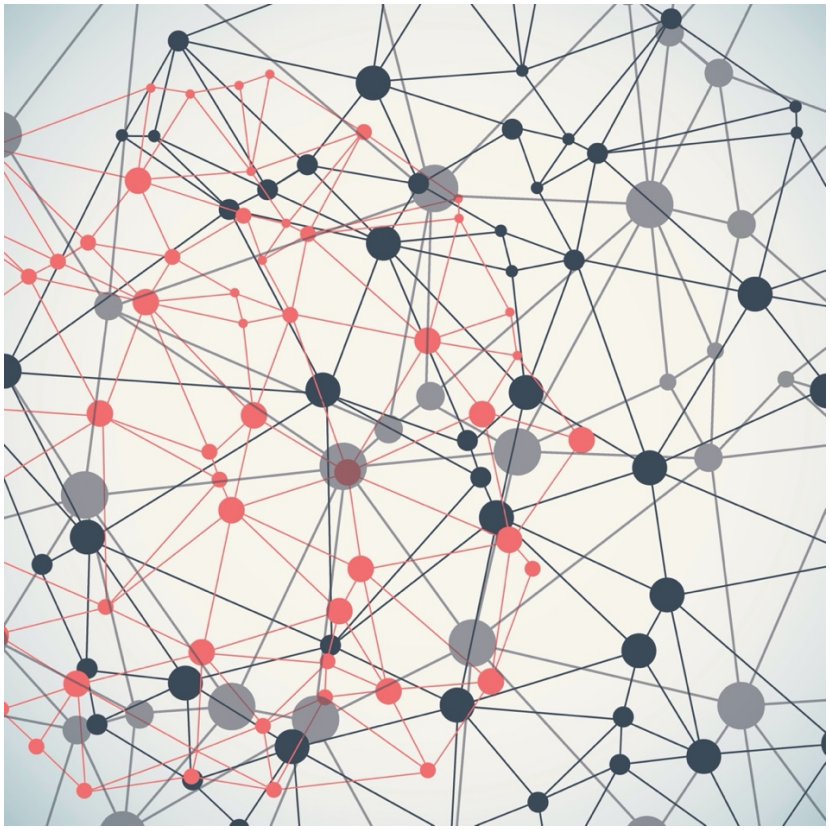
Big Data Technologies Coursework



Master Course: Advanced Cyber Security

Module Name: Big Data Technologies (7CCSONBD)

Student Group 'F':

1. Olga Suzuki
2. Mohamed Abozeid
3. Syed Navaid Shamsee

Task 1 - MapReduce combiners

Introduction

MapReduce is a programming model designed for processing and generating large data sets with a parallel, distributed algorithm on a Hadoop cluster. It involves three key stages: mapping, shuffling, and reducing, which end in producing the final output.

During the mapping stage, the data chunks provided by the distributed file system are transformed into key-value pairs according to user-defined code.

Next, the key-value pairs undergo the shuffling phase, where they are grouped by keys and then passed to the reducer. Each reducer receives all values associated with a particular key.

In the reducing stage, the reducer performs aggregation and transformation tasks on the data as specified by the user-defined code.

Once all reduce tasks are completed, the MapReduce framework assembles the final combined output.

A combiner, often referred to as a "mini-reducer," optimizes the process by pre-aggregating values at the mapper level. This pre-aggregation reduces the amount of data shuffled and speeds up the reducer tasks. Each combiner operates on the output of a single mapper, grouping keys and values before they are shuffled and sent to the reducers.

The diagrams below illustrate examples of a MapReduce process calculating the sum of all grades per student. Diagram 1 demonstrates the process without the combiner function, while Diagram 2 includes the combiner function.

Diagram 1: MapReducer without combiner - calculate sum of all grades

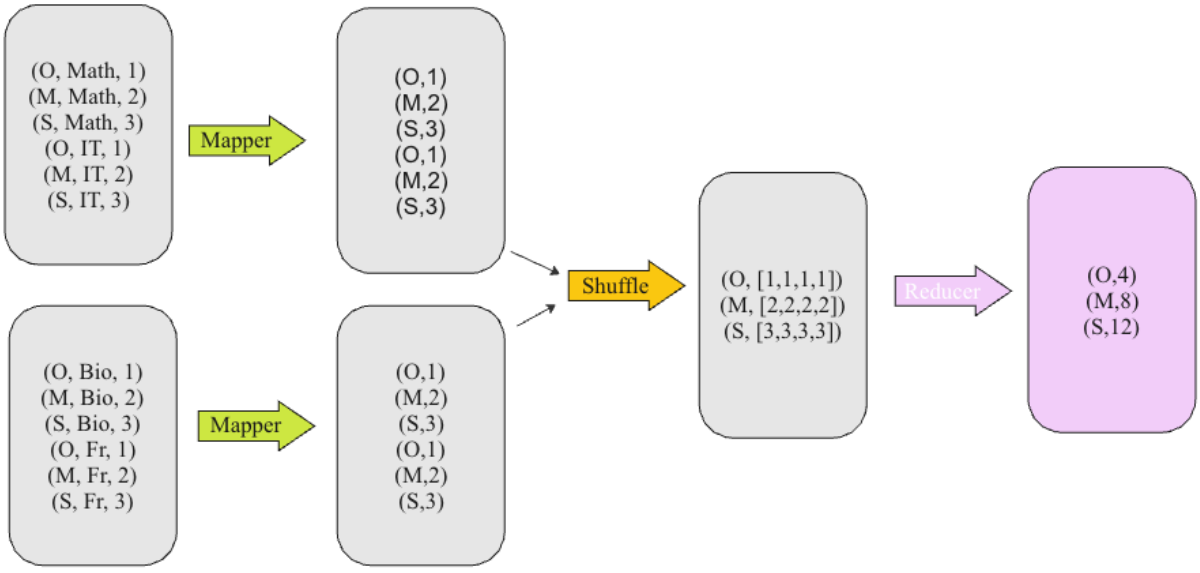
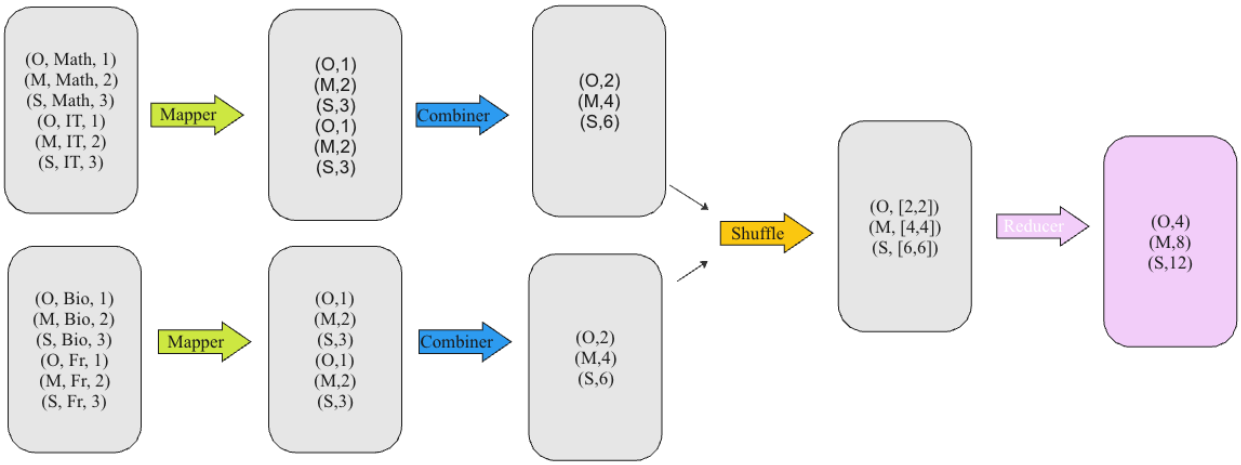


Diagram 2: MapReducer with combiner - calculate sum of all grades



Because the sum function is both commutative and associative, the reducer calculates the results correctly, whether a combiner is used.

Commutative means that changing the order of the inputs does not change the result, i.e., $a+b=b+a$

Associative means that the grouping of the inputs does not affect the result, i.e., $(a+b)+c=a+(b+c)$

Example of function not working correctly with the combiner

We will use the average function to demonstrate that a combiner does not produce correct results when applied to a function that is neither commutative nor associative.

The average function is neither commutative nor associative because its output depends on the number of elements being averaged. When elements are grouped differently, the output changes.

For example, the average of *(1,2,3)* is *2*, while the average of *(1,5)* is *3*.

Program task1_c1.py - mapper and reducer only

Source code of task1_c1.py

The following program calculates and an average student grade without the combiner function:

```
from mrjob.job import MRJob
import statistics

class StudentAVGGradeCalculator(MRJob):

 def mapper(self, _, value):
    data = value.split(',')
    if len(data) >= 3:
       student = data[0].strip()
       grade = data[2].strip()
       yield student, int(grade)

 def reducer(self, key, values):
    yield key, statistics.mean(values)

if __name__ == '__main__':
     StudentAVGGradeCalculator.run()
```

Note: we intentionally removed the comments from the code for the report. Comments can be viewed in the code submitted with the report

Output of task1_c1.py

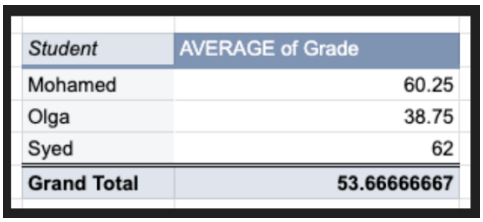
To execute the program, we need to run the following command:

python task1_c1.py student_grades.txt

When program finishes, we get the following output:

```
Running step 1 of 1...
job output is in /var/folders/1k/sxr05tz179s64bdq4w84qxpw0000gn/T/task1_c1.olga.2024060
1.121410.489960/output
Streaming final output from /var/folders/1k/sxr05tz179s64bdq4w84qxpw0000gn/T/task1_c1.o
lga.20240601.121410.489960/output...
"Syed"  62
"Olga"  38.75
"Mohamed"       60.25
```

We can verify that the average was calculated correctly by calculating the average in the google sheet or excel document:



Student	AVERAGE of Grade
Mohamed	60.25
Olga	38.75
Syed	62
Grand Total	**53.66666667**

Program task1_c2.py mapper, combiner, and reducer

The following program calculates and an average student grade with the combiner function:

Source code of task1_c2.py

```python
from mrjob.job import MRJob
import statistics
from mrjob.step import MRStep
class StudentAVGGradeCalculator(MRJob):
    def mapper(self, _, value):
        data=value.split(',')
        student=data[0].strip()
        grade=data[2].strip()
        if len(data)>=3:
         yield student, int(grade)

    def combiner(self, key, values):
        yield key, statistics.mean(values)


    def reducer(self, key, values):
        yield key, statistics.mean(values)


    def steps(self):
        return[
        MRStep(mapper=self.mapper,
        combiner=self.combiner,
        reducer=self.reducer),
    ]
```

```
if __name__ == '__main__':
    StudentAVGGradeCalculator.run()
```

Note: we intentionally removed the comments from the code for the report. Comments can be viewed in the code submitted with the report

Output of task1_c2.py

To execute the program, we need to run the following command:

python task1_c2.py student_grades.txt

When program finishes, we get the following output:

```
Running step 1 of 1...
job output is in /var/folders/1k/sxr05tz179s64bdq4w84qxpw0000gn/T/task1_c2.olga.20240601.122227.297029/output
Streaming final output from /var/folders/1k/sxr05tz179s64bdq4w84qxpw0000gn/T/task1_c2.olga.20240601.122227.297029/
output...
"Syed"  62.27272727272727
"Olga"  40.388888888888886
"Mohamed"       61.92307692307692
```

As we can see, the output is not correct. This occurs because the average function is neither commutative nor associative. As previously explained, the combiner groups the inputs before passing the keys and values to the reducer, which leads to incorrect results.

Task 2 – MongoDB queries

Task 2.1

Load the file championsleague_1.json into a database cw2 and a collection cl.

Loading the file

The file can be loaded with the following script:

```
mongoimport --db cw2 --collection cl --drop --file championsleague_1.json
```

Command Output

Following output shows that file is imported successfully:

```
NSHAMSEE-M-NXX7:~ nshamsee$ mongoimport --db cw2 --collection cl --drop --file championsleague_1.json
2024-05-24T20:53:04.094+0400        connected to: mongodb://localhost/
2024-05-24T20:53:04.095+0400        dropping: cw2.cl
2024-05-24T20:53:04.638+0400        23285 document(s) imported successfully. 0 document(s) failed to import.
NSHAMSEE-M-NXX7:~ nshamsee$
```

Task 2.2

Write a query that returns the name, number of followers, and number of friends, of each user with fewer than 25 friends, whose name starts with "A" (case insensitive) and ends with "es" (case sensitive). The results must be sorted in decreasing order of displayName

The Query

```
db.cl.find(
{
"displayName": { $regex: /^[Aa].*es$/ }, // Matches names starting with "A" (case
insensitive) and ending with "es" (case sensitive)
"friendsCount": { $lt: 25 } // Filters users with fewer than 25 friends
},
{
"displayName": 1,
"friendsCount": 1,
"followersCount": 1
}
).sort({ "displayName": -1 });
```

Query results

```
[
 {
  _id: ObjectId('665144158d7f119be698c284'),
  displayName: 'angie torres',
  friendsCount: 23,
  followersCount: 33
 },
 {
  _id: ObjectId('665144158d7f119be698c28e'),
  displayName: 'angie torres',
  friendsCount: 23,
  followersCount: 33
 },
 {
  _id: ObjectId('665144178d7f119be698f209'),
  displayName: 'Arizona Companies',
  friendsCount: 0,
  followersCount: 10
 },
 {
  _id: ObjectId('665144178d7f119be698e20b'),
  displayName: 'Adejies',
  friendsCount: 13,
  followersCount: 10
 },
 {
  _id: ObjectId('665144178d7f119be698f247'),
  displayName: 'Adejies',
  friendsCount: 13,
  followersCount: 10
 },
 {
  _id: ObjectId('665144178d7f119be698f751'),
  displayName: 'Adejies',
  friendsCount: 13,
  followersCount: 10
 }
]
```


Task 2.3

Write a query that returns the average number of followers, over all documents, where the users have more than 100000 friends

The Query

```
db.cl.aggregate ( [
```

```
    { $match: {"friendsCount": {$gt: 100000 } }},  // Match documents where
friendsCount is greater than 100000
    { $group: { _id:0, Average: { $avg: '$followersCount'}  }   }
] )
```

Query results

```
[{_id: 0, Average: 581728.3636363636 }]
```

Task 2.4

Write a query that returns the average ratio between numbers of followers and number of friends, over all documents. Note that the number of followers must be >0 for the ratio to be defined.

The query:

```
db.cl.aggregate([
{
$match: {
followersCount: { $gt: 0 }, // Match documents where number of followers is greater
than 0
friendsCount: { $gt: 0 } // Match documents where number of followers is greater than
0
}
},
{
$project: {
ratio: { $divide: ["$followersCount", "$friendsCount"] } // Calculate the ratio of
followers to friends for each document
}
},
{
$group: {
_id: null, // Group all documents together
averageRatio: { $avg: "$ratio" } // Calculate the average ratio
}
},
{
$project: {
averageRatio: 1
}
}
]);
```

Query results:

```
[
{
"_id": null,
"averageRatio": 156.5685824143264
}
]
```


Task 2.5

Write a query that returns the number of users who have at least 1000 friends, posted a tweet (the field verb has the value "post") and whose tweet (field body) contains the string “Madrid” (even as part of a word).

The query:

```
db.cl.aggregate([
{
$match: {
friendsCount: { $gte: 1000 }, // At least 1000 friends
verb: "post", // Posted a tweet
body: { $regex: /Madrid/ } // Body contains the string "Madrid" (case sensitive)
}
},
{
$count: "numberOfUsers" // Count the number of matching documents
}
]);
```

Query results:


```
[
{
"numberOfUsers": 189
}
]
```


Task 2.6

Write a query that displays the number of followers of users who have more than 200 and fewer than 203 statuses (the number of statuses is in statusesCount).

The query:

```
db.cl.find(
{
statusesCount: { $lt: 203,$gt: 200 },
},
{
followersCount:1
})
```

Query results:

```
[
 { _id: ObjectId('665144158d7f119be698c32a'), followersCount: 139 },
 { _id: ObjectId('665144168d7f119be698c856'), followersCount: 16 },
 { _id: ObjectId('665144168d7f119be698cb0f'), followersCount: 7 },
 { _id: ObjectId('665144168d7f119be698cc2c'), followersCount: 1056 },
 { _id: ObjectId('665144168d7f119be698cee6'), followersCount: 23 },
 { _id: ObjectId('665144168d7f119be698cee8'), followersCount: 23 },
 { _id: ObjectId('665144178d7f119be698dee3'), followersCount: 79 },
 { _id: ObjectId('665144178d7f119be698f02c'), followersCount: 25 },
 { _id: ObjectId('665144178d7f119be698f084'), followersCount: 322 },
 { _id: ObjectId('665144178d7f119be698f7a2'), followersCount: 77 },
 { _id: ObjectId('665144188d7f119be699159d'), followersCount: 344 }
]
```

Task 3: Apache Spark Analysis of User Occupations by Age Group

This task aims to analyze user occupation data using Apache Spark. The dataset provided in *u.user* contains records of users, each with five attributes: *user ID, age, gender, occupation, and zipcode.* Each record is formatted as a pipe-separated **" | "** string. for example, *1|24|M|technician|85711*.

The goal is to process this data and identify the **10** most frequent occupations for users within two specific age groups: *40-50* and *50-60.*

To achieve this, we will first parse the u.user file to extract the relevant attributes for each user. Using the Spark framework, we will filter the users based on their age groups and compute the frequency of each occupation within the specified age ranges. The final output will display the 10 most common occupations and their frequencies for both age groups.

Program task3.py

The following parses an input file and then filters the data based on the age group. Later, it will compute the frequency of each occupation within the 40-50 age group and 50-60 age group as well.

Code snippet:

```
from pyspark import SparkContext
from operator import add

"""
Initialize SparkContext
"""

sc = SparkContext('local', 'pyspark')

"""
      Define age_group Function
"""
def age_group(age):
"""
      Assigns an age group label based on the age.
"""
       if age < 10:
              return '0-10'
       elif age < 20:
       return '10-20'
```

```
        elif age < 30:
        return '20-30'
        elif age < 40:
        return '30-40'
        elif age < 50:
        return '40-50'
        elif age < 60:
        return '50-60'
        elif age < 70:
        return '60-70'
        elif age < 80:
        return '70-80'
        else:
        return '80+'

def parse_with_age_group(data):

"""
        Parse each line of the input data and categorizes it into the correct age
group.
        The expected format for the input line is:

                        userid|age|gender|occupation|zipcode
"""
userid, age, gender, occupation, zipcode = data.split('|')

"""
        The return array will be as following:
                        userid|age_group|gender|occupation|zipcode
"""
return userid, age_group(int(age)), gender, occupation, zipcode, int(age)

# Load data from u.user file
fs = sc.textFile("file:///Users/kamal/Kamal-Drive/MSc/Modules/Big Data
Technologies/Coursework/Coursework-OSK/big_data_mrjob/assigment/task3/u.user")

"""
        Apply the parse_with_age_group function to each record. a new RDD fs2 is now
created with the mapped age-group
"""
fs2 = fs.map(parse_with_age_group)

# Filter the records to include only those in the 40-50 age group
fs4050 = fs2.filter(lambda x: x[1] == "40-50")

# Filter the records to include only those in the 50-60 age group
fs5060 = fs2.filter(lambda x: x[1] == "50-60")

# Extract the distinct occupations list for the 40-50 age group
distOccupation_4050 = fs4050.map(lambda x: x[1]).distinct()
```

```
# Extract the distinct occupations list for the 50-60 age group
distOccupation_5060 = fs5060.map(lambda x: x[1]).distinct()

# Create key-value pairs with (occupation, 1)
occupation_4050 = fs4050.map(lambda x: (x[3], 1))
occupation_5060 = fs5060.map(lambda x: (x[3], 1))

# Reduce by key to count the occurrences of each occupation
occupation_count_4050 = occupation_4050.reduceByKey(add)
occupation_count_5060 = occupation_5060.reduceByKey(add)

# Collect the results and sort by frequency in descending order
result_4050 = occupation_count_4050.sortBy(lambda x: -x[1]).collect()
result_5060 = occupation_count_5060.sortBy(lambda x: -x[1]).collect()

# Print the top 10 results for occupation counts in the 40-50 age group
print("Top 10 occupations in age group 40-50:")
for item in result_4050[:10]:
       print(item)

# Print the top 10 results for occupation in the 50-60 age group
print("Occupations in age group 50-60:")
for item in result_5060[:10]:
       print(item)

"""
Stop SparkContext
"""
sc.stop()
```

To run the application, task3.out is going to be generated

spark-submit task3.py > task3.out

Output of Program task3.py

Task3.out:

```
kamal@Mohameds-MBP task3 % cat task3.out
Top 10 occupations in age group 40-50:
('educator', 25)
('other', 22)
('administrator', 20)
('engineer', 14)
('librarian', 13)
('writer', 13)
('executive', 11)
('programmer', 10)
('scientist', 8)
('healthcare', 7)
Occupations in age group 50-60:
('educator', 23)
('administrator', 12)
('other', 11)
('librarian', 11)
('writer', 6)
('marketing', 5)
('engineer', 5)
('lawyer', 3)
('healthcare', 3)
('programmer', 3)
```

Contribution

Each member of the team equally contributed to the coursework. We utilized Microsoft Teams for brainstorming sessions and to clarify any doubts and made sure that we have a clear understanding of the tasks. Each one of us wrote our programs and queries independently while discussing and sharing our thoughts on Microsoft team. Finally, we combined our work after the group discussions and refinements. Every team member played a role in contributing to this document, working together seamlessly through SharePoint for document collaboration.